

Task 7 – Analyse and Optimise

For Task 7, the Tripistry database was populated with a large amount of test data to make optimisation meaningful. The following data was generated:

Table	Approximate Rows
traveller	1000
travel_agency	10
package	500
service	1000
books	50 000
review	50 000

Query 1 – Traveller Bookings and Packages

```
SELECT T.Fname, T.Lname, P.Name AS Package_Name
FROM books B
JOIN traveller T ON B.User_id = T.User_id
JOIN package P ON B.Package_id = P.Package_id
WHERE P.Price BETWEEN 3000 AND 7000;
```

This query retrieves travellers and the packages they booked within a certain price range.

Optimisation Applied:

```
CREATE INDEX idx_package_price_packageid ON package(Price, Package_id);
```

Stage	EXPLAIN Rows	Rows Returned	Execution Time
Before Optimisation	3	10700	0.051 sec
After Optimisation	3	10700	0.024 sec

The execution time improved because the new index allowed MariaDB to filter packages by price more efficiently before performing the joins.

Query 2 – Average Package Ratings

```
SELECT P.Name, AVG(R.Rating) AS Average_Rating
FROM package P
JOIN review R ON P.Target_id = R.Target_id
GROUP BY P.Name;
```

Optimisations Applied:

```
CREATE INDEX idx_review_target_rating ON review(Target_id, Rating);
```

```
CREATE INDEX idx_package_target_name ON package(Target_id, Name);
```

Stage	EXPLAIN Rows	Rows Returned	Execution Time
Before Optimisation	12	500	0.176 sec
After Optimisation	15	500	0.070 sec

This optimisation improved performance significantly because the database could locate review rows faster and perform grouping more efficiently.

Conclusion

The optimisation process demonstrated that indexes can improve the performance of large database queries, especially queries involving joins, filtering and grouping. The second query showed the biggest improvement, decreasing execution time by approximately 60%.